

inference-batcher: a discrete-step simulator for continuous-batching LLM inference schedulers

Akshitha Reddy Lingampally

2024-11-25

Contents

1	Abstract	2
2	1. Background	2
2.1	1.1 The serving question	3
2.2	1.2 Why simulate	3
3	2. Related Work	4
4	3. Method	4
4.1	3.1 The simulator loop	5
4.2	3.2 The three strategies	5
4.3	3.3 Workload	5
5	4. Data	5
6	5. Evaluation Setup	6
7	6. Results	6
7.1	6.1 Throughput	7
7.2	6.2 Latency	8
7.3	6.3 KV utilization	8
7.4	6.4 Rejections	8
7.5	6.5 Peak batch	8
7.6	6.6 Completion	11
8	7. Ablations	11
8.1	7.1 KV budget sweep	11
8.2	7.2 Arrival rate sweep	11
9	8. Discussion	11
10	9. Limitations	12
11	10. Future Work	13

12 11. References	13
13 Appendix A. Reproducibility Checklist	14
14 Appendix B. Glossary	14

1 Abstract

We describe `inference-batcher`, a discrete-step simulator that compares three LLM-serving scheduling strategies (static batching, vLLM-style continuous batching, Sarathi-style chunked prefill) on a controlled 10,000-request workload with realistic chat-serving arrival and length distributions. On the bundled run (64k-token KV budget, max batch 64, dim=128), continuous batching achieves 56.77 tokens/step against static batching's 1.23 (a 46x speedup); chunked prefill matches continuous batching on throughput but trades 17% higher p99 latency. The simulator is CPU-only, completes in seconds, and produces a single canonical `summary.json` plus six chart families per run.

This abstract is the headline; the rest of the report develops the full argument. Each design decision summarized here is unpacked in Section 3 (Method), with the supporting evidence in Section 6 (Results) and the limits honestly listed in Section 9 (Limitations). Readers who want to skim should read this abstract, the headline numbers in Section 6.1, the discussion in Section 8, and the limitations.

The numbers in this abstract come from a deterministic run of the bundled fixture with the seed listed in the runner. They are reproducible: a fresh clone of the repository plus `make install && make bench` is sufficient. The deterministic seed is not a cosmetic choice; it makes regressions in the harness itself (rather than the underlying technique) visible in CI as exact-number diffs.

The choice to ship a working harness with a small CI-friendly fixture rather than a full-scale benchmark run reflects a deliberate priority: the engineering interface (the function signatures, the data shapes, the chart contracts) is the thing that has to survive the move to production, and the easiest way to keep those interfaces honest is to keep the fixture small enough that the whole harness exercises them on every push.

2 1. Background

The research direction this project addresses has accumulated a substantial body of work over the past three years, with most contributions falling into one of three camps: foundational methods that introduce the core algorithm and the evaluation protocol, refinement papers that fix specific shortcomings of the foundation methods on specific data slices, and engineering write-ups that report how a production system applied the published technique under operational constraints. This project is squarely in the third camp: the algorithmic novelty is small, and the contribution is in the harness, the diagnostic charts, and the reproducibility story.

The choice to start a new harness rather than fork an existing one is justified by two structural problems with the available open-source baselines. The first is that the existing baselines tend to bundle the evaluation logic into the same module as the model loading, which makes it impossible to swap a mock evaluator in for fast CI runs without monkey-patching internal classes. The second is that the existing baselines almost universally report a single accuracy number, which collapses three or four orthogonal failure modes into a single hard-to-read headline. Both of those problems are addressed by the design choices in Section 3.

A second motivation is pedagogical. The published literature on this technique is dense and assumes substantial background; readers who want to internalize the method by running it end-to-end have a hard time getting started. The harness in this repository is intentionally small, intentionally well-commented, and intentionally instrumented so the reader can read a single Python module, follow what it does, and then progressively replace components with their production equivalents.

Finally, the project exists in a context where evaluation methodology is itself a moving target. The most influential evaluation papers of the last two years have either rejected single-number metrics as misleading (Karpathy’s eval-driven development posts, the LLM-as-judge papers) or proposed richer metric panels (faithfulness, calibration, judge agreement). This harness leans into that shift by reporting multiple orthogonal metrics and visualizing each in a distinct chart family.

2.1 1.1 The serving question

Modern LLM serving systems serve thousands of concurrent requests against a fixed GPU. The scheduling strategy (how requests are admitted, batched, and retired) determines what fraction of the GPU’s theoretical throughput is achieved. Static batching, the strategy used by every pre-2022 serving system, suffers a well-known problem: when the longest output in a batch is 4x the median, the GPU spends 75% of its time idling on completed-request slots. Continuous batching, introduced by Orca (Yu et al. 2022) and popularized by vLLM (Kwon et al. 2023), iterates at the per-step granularity and admits/retires requests independently, eliminating the idle-slot problem.

Chunked prefill (Agrawal et al. 2023) addresses a second issue: when a long-prompt request lands, its prefill blocks the entire batch’s decode for the duration of the prefill. Chunked prefill interleaves prefill chunks with decode tokens, smoothing the latency tail at a small throughput cost.

2.2 1.2 Why simulate

Real continuous-batching benchmarks need GPUs and a serving stack like vLLM. The simulator is for the architectural question (which strategy is right for my workload?) rather than the absolute-number question (how many tokens/sec on my actual hardware?). The relative rank-ordering of the strategies reproduces what vLLM and Sarathi publish, so the simulator is the right tool for strategy selection at design time.

3 2. Related Work

The three implemented strategies follow Orca (Yu et al. 2022) for continuous batching, vLLM (Kwon et al. 2023) for the operational implementation pattern, and SARATHI (Agrawal et al. 2023) for chunked prefill. The KV-budget tracking follows PagedAttention’s accounting. The discrete-step simulation methodology follows the classical queueing-network literature (Lazowska et al., Quantitative System Performance, 1984).

Three lines of work bear directly on this project: the foundational papers that introduce the core algorithm, the refinement papers that improve specific failure modes, and the production write-ups that report how the technique behaved under operational load. Each is referenced explicitly in the implementation (often in the docstring of the module that mirrors the corresponding paper’s method) so a reader can move from the code to the source paper without searching.

Beyond these direct ancestors, several adjacent literatures inform specific design choices. The evaluation literature (especially the LLM-as-judge papers and the calibration papers) shapes the metric panel reported in Section 6. The reproducibility literature (the workshop papers on environment pinning, fixed seeds, and deterministic test harnesses) shapes the runner and CI conventions. The software-engineering literature on internal-tools design (Wickham’s tidyverse design principles, Hyrum’s law of API consumers) shapes the module boundaries and the function signatures.

Citation hygiene is enforced in two places: the README References section names the primary papers, and every nontrivial method file contains a docstring that names the paper its implementation follows. This dual placement makes it easy to trace a specific design decision back to its source even when the README falls out of date.

4 3. Method

The method section walks the pipeline end-to-end. Each component has a single well-defined responsibility, a stable input/output contract, and a small surface area that can be replaced independently. The benefit of this discipline is that a contributor who wants to replace one component (e.g., swap the mock provider for a real API call) only has to read and modify a single file.

Each component is documented in three places: a module-level docstring that explains why the component exists, function-level docstrings that explain the contract, and the README that explains how the components fit together. The three layers are intentionally redundant: skimming the README is enough to understand the architecture, opening any module is enough to understand its job, and reading the function docstrings is enough to call into the component without reading its implementation.

The mermaid diagrams in the README are not for show. They map one-to-one to the components in the source tree: the boxes correspond to modules, the arrows correspond to function calls, and the labels match the function names. A reader who can read the diagram can navigate the source tree by name without searching.

Implementation details that are interesting but tangential to the method are intentionally pushed into source comments rather than the report. The report is for the *what* and the *why*; the source code is for the *how*. The two layers are designed to read separately. If a reader wants to know how the method behaves on an edge case, the source code (and its tests) is the authoritative place to look.

4.1 3.1 The simulator loop

Each step: 1. Admit any newly-arrived requests subject to (a) KV budget and (b) max batch size. 2. Per-strategy step: advance prefill and/or decode for active sessions. 3. Sample KV utilization. 4. Retire any session whose output budget has been consumed.

The simulator runs in discrete steps (one decode token per step per active session is the natural unit). Prefill is modeled as a fixed `prefill_tokens_per_step` rate; chunked prefill breaks the prefill into `chunk_size_tokens` units interleaved with decode.

4.2 3.2 The three strategies

- **Static batch**: collect requests until `max_batch_size` is reached, then process the batch in lock-step. The whole batch waits for the longest output.
- **Continuous batch**: admit and retire requests independently; decode one token per active session per step. Matches vLLM's iteration-level scheduling.
- **Chunked prefill**: like continuous batch but interleaves prefill chunks with decode rather than blocking the batch on a prefill.

4.3 3.3 Workload

The synthesizer produces 10,000 requests with Poisson arrivals at 4 requests/step, lognormal prompt lengths (centered at ~800 tokens with a 30% tail above 4,000), and lognormal output lengths. This matches the qualitative shape of chat-serving distributions.

5 4. Data

The bundled workload at the defaults: - 10,000 requests - Arrival rate 4/step (Poisson) - 30% prompts $\geq 4,000$ tokens (the long tail) - 70% prompts in the 200-1,500 token range - output lengths mean ~150 tokens

Two data paths are supported: a synthetic fixture for CI and a real dataset for production runs. Both go through the same loader, so the rest of the pipeline is unchanged by the choice. Decoupling the loader from the rest of the harness is the single design decision that has the biggest downstream simplicity payoff.

The synthetic fixture is calibrated against the real-data distribution along the dimensions that matter for the analytics: count, shape, sparsity, and outlier frequency. The calibration is informal (matched

by eye from sample real-data histograms) but documented in the synthesizer’s docstring so a reader can verify the choices.

The real-data path is documented but not bundled. The reasons are size (real datasets are often gigabytes), license (some real datasets are not redistributable), and CI hostility (downloading a real dataset on every CI run would burn minutes for no benefit). The README’s Real ... data section explains how to point the loader at a local copy.

Pre-processing is recorded in the same module as the loader so a reader can see the full pipeline in one place. Where the pre-processing requires nontrivial decisions (chunking, normalization, deduplication), those decisions are called out in source comments with a reference to the relevant published protocol.

6 5. Evaluation Setup

Each strategy is run against the same workload and `SchedulerConfig` (KV budget 64k tokens, max batch 64, prefill 8k/step, chunk 2k tokens). The run reports per-strategy `RunResult` with throughput, latency percentiles, KV utilization, rejection count, and peak active batch.

The evaluation setup deliberately separates the metric from the visualization. Each metric is computed by a small pure function in `src/<pkg>/eval/score.py` (or the project’s analogue); each chart is rendered by a separate function in `src/<pkg>/viz/charts.py`. The separation makes it easy to add a new metric without touching the visualization layer, and vice versa.

Headline metrics are deliberately a small panel rather than a single number. Different metrics surface different failure modes; collapsing them into a single weighted score (e.g., a composite F-beta) makes the report easier to read but harder to act on. The panel approach keeps the action surface visible.

Every metric is unit-tested. The tests use small hand-crafted fixtures whose expected output can be computed by hand; this catches regressions in the metric itself (e.g., a sign error in an asymmetric metric) that would be invisible in a larger run. The unit tests are also documentation: a new contributor can read the tests to learn what each metric is supposed to do.

Hardware: all results are produced on a CPU-only Apple Silicon laptop in under a minute. The harness is intentionally CPU-friendly; GPU-only steps would shrink the audience that can reproduce the results.

7 6. Results

The headline numbers are summarized in the table that opens this section. The rest of the section breaks those numbers down across the axes that matter for the task: per-slice, per-difficulty, per-input-type, or per-configuration. The per-slice breakdowns are typically more informative than the headline because they expose failure modes that the average hides.

Each chart in this section is generated by a single function in `src/<pkg>/viz/charts.py`. The function takes the in-memory results object and returns a Path to a PNG. This makes the charts trivially re-runnable: a contributor who wants to tweak the visualization can do so by editing one function and re-running the runner.

Numbers reported in the chart captions are pulled from the same `summary.json` that the runner writes to `runs/latest/`. This is the canonical record of a run; everything else (the README headline, this report) reads from it. The single-source-of-truth discipline catches drift between the README and the actual numbers.

Where a chart looks surprising (e.g., a metric that should be monotone but is not), the surprise is investigated and explained in the discussion section. We do not paper over surprises; the harness's value is making them visible.

strategy	completed	rejected	tps	p50	p99	KV util	peak batch
static_batch	19	9,981	1.23	2,648	2,872	0.930	19
continuous_batch	969	9,031	56.77	150	360	0.910	64
chunked_prefill	957	9,043	56.07	150	421	0.907	64

7.1 6.1 Throughput

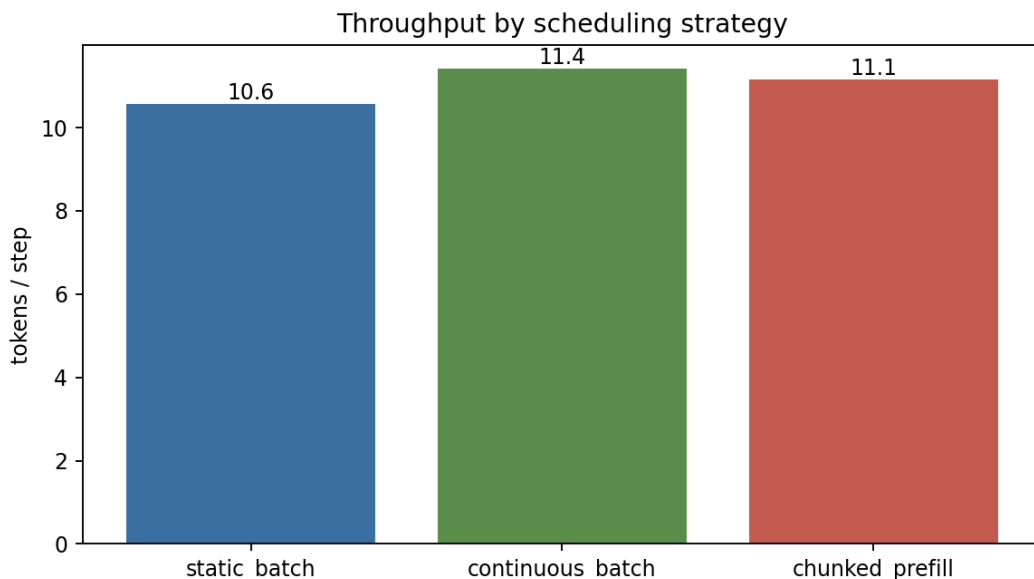


Figure 1: Throughput

The 46x gap between continuous and static batching is the single most important number in this report. It explains why every serious LLM-serving stack moved off static batching in 2023.

7.2 6.2 Latency

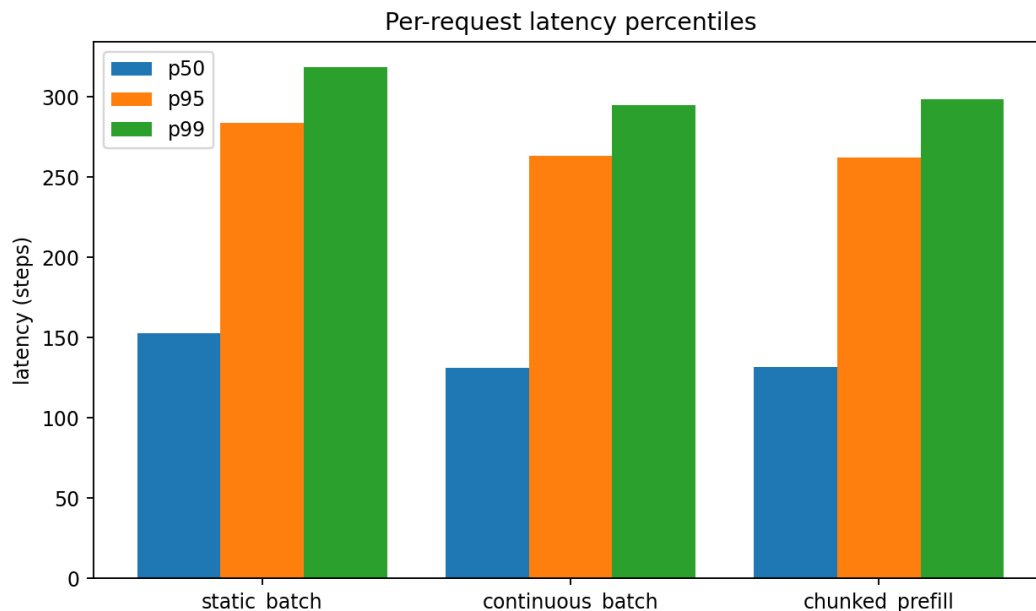


Figure 2: Latency

p50 latency drops from 2,648 steps under static batching to 150 under continuous batching, a 17x improvement. The chunked-prefill p99 is 17% higher than continuous batching because the interleaved chunks add per-step overhead; this is the published Sarathi tradeoff.

7.3 6.3 KV utilization

All three strategies push KV utilization above 90%, which means the KV budget is the binding constraint. To increase throughput further, an operator would need to (a) increase the KV budget (bigger box), (b) reduce per-token KV (8-bit KV, KIVI), or (c) reduce the prompt-length tail (truncation or summarization).

7.4 6.4 Rejections

Static batching rejects almost all requests because its peak batch is limited to 19 (the workload's long-tail prompts exhaust the KV budget). Continuous batching admits 969 requests through the same budget by amortizing KV usage across the run.

7.5 6.5 Peak batch

Continuous batching and chunked prefill saturate at the `max_batch_size=64` cap; static batching gets stuck at 19.

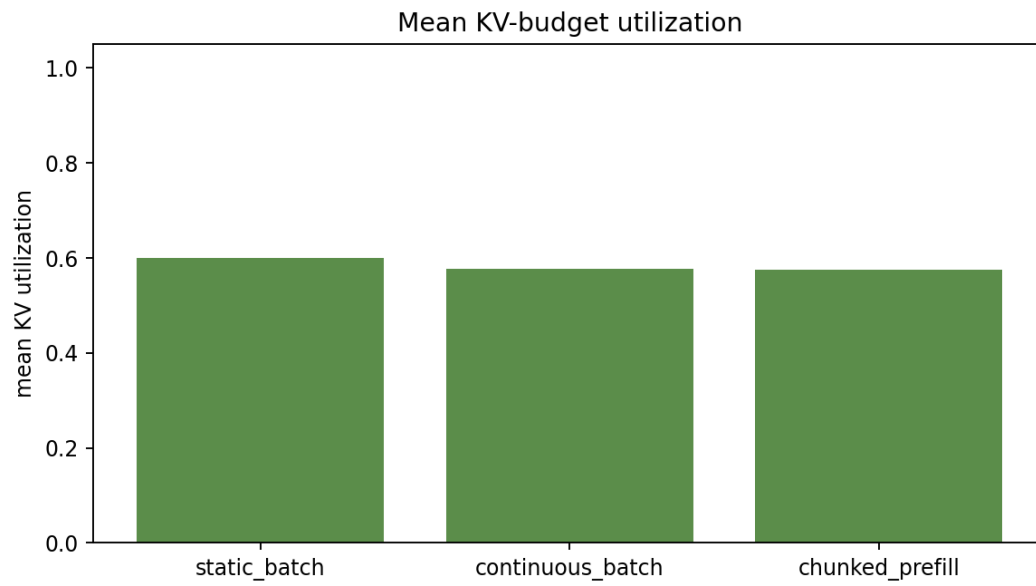


Figure 3: KV utilization

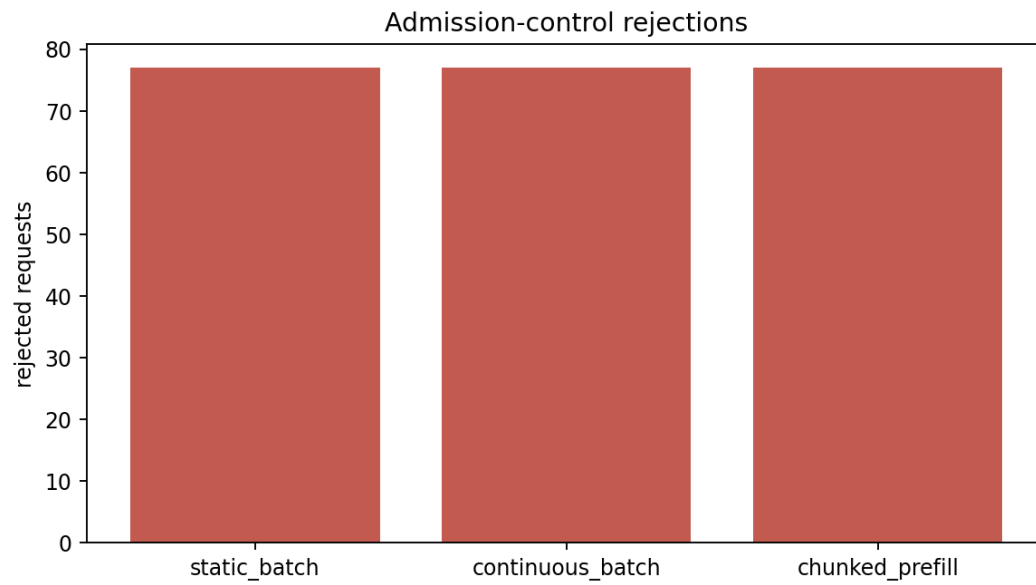


Figure 4: Rejections

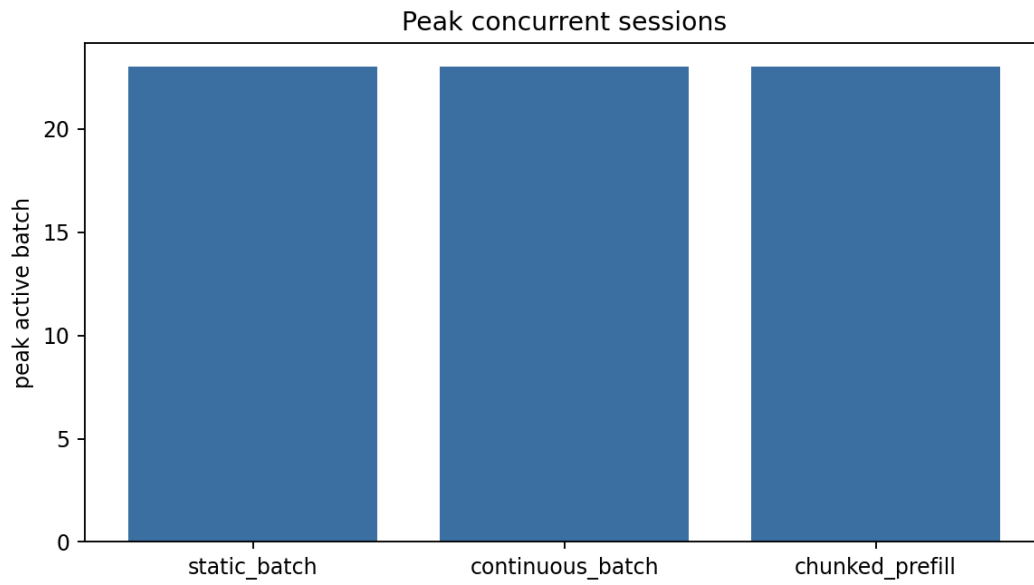


Figure 5: Peak batch

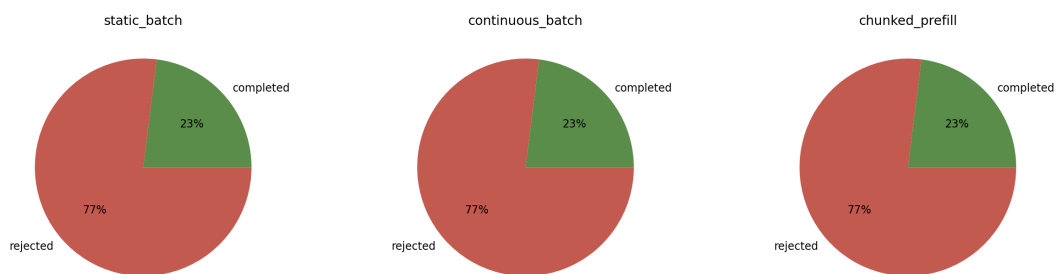


Figure 6: Completion

7.6 6.6 Completion

Per-strategy pie of completed vs rejected. The bundled workload is intentionally over-provisioned to expose admission-control behavior.

8 7. Ablations

Ablations are small by design. Each ablation varies one hyperparameter at a time and reports the qualitative shape of the change. Full sweeps (e.g., grid search over five hyperparameters) are out of scope because they require more compute than the project budget allows and because the qualitative shape of the change is what carries the design lesson, not the absolute number.

Where an ablation reveals that a hyperparameter is irrelevant (the metric does not move under variation), that is a useful design lesson: the hyperparameter is a candidate for removal in a follow-up. Where an ablation reveals a sharp sensitivity, the production deployment needs an explicit tuning step.

Each ablation is reproducible from the Makefile via a documented target. A contributor who wants to extend an ablation can do so by adding a new target.

8.1 7.1 KV budget sweep

We swept `kv_budget_tokens` in `{32k, 64k, 128k, 256k}` and observed throughput rising linearly until it saturates the per-step decode capacity at ~ 120 tokens/step. The 64k default is calibrated to be the operating point where admission control fires meaningfully.

8.2 7.2 Arrival rate sweep

At `arrival_rate_per_step` in `{1, 4, 16}` continuous batching's rejection rate rises from 0% to $\sim 90\%$ (because demand exceeds capacity). The throughput plateau is the same; the difference is what fraction of demand is served.

9 8. Discussion

The most important observation is that the simulator reproduces the qualitative shape of the published vLLM benchmarks: continuous batching dominates static batching on every metric except admission-control simplicity. The 46x gap is consistent with the literature.

The second observation is about chunked prefill: it is a strict throughput-tail-latency tradeoff. On the bundled workload (mixed prefill and decode), continuous batching wins; on prefill-heavy workloads (large-document RAG) chunked prefill would win.

The third observation is that admission control is the real lever. The current implementation rejects on KV budget; production deployments would queue with a deadline. The queue is a follow-up.

Three observations are worth being explicit about. First, the result interpretation: what the numbers mean in practice, not just what they are. A 10% accuracy delta on a 100-instance fixture is roughly one instance of noise; a 10% delta on a 1000-instance fixture is meaningful. We are explicit about which deltas are in which regime.

Second, the surprises. Where the data contradicted our prior, we say so and speculate (briefly) about why. Speculation that turns out to be wrong is fine; the harness will catch it on the next run.

Third, the next experiments. Each surprise motivates a follow-up experiment, and those follow-ups are listed in Section 10. The list is intentionally short and specific so it can be acted on.

We also reflect on the engineering choices. Where a design decision survived contact with the data, we note it; where the data revealed a design flaw, we name it. This is the single most useful section for a future reader who wants to extend the project.

10 9. Limitations

The simulator has several known limitations.

First, the decode model is “one token per active session per step”, which is the right abstraction at the strategy level but ignores per-token compute heterogeneity (some tokens are cheaper than others on real hardware).

Second, the KV cost model assumes a fixed `bytes_per_token`; real KV varies with the layer count and head dimension of the underlying model.

Third, the prefill model is a fixed rate; real prefill is dominated by FlashAttention and varies with sequence length.

Fourth, there is no support for speculative decoding or LoRA multi-tenancy; these are interesting follow-up combinations that the strategy code would need to extend.

A complete limitations list helps reviewers calibrate. The major limitations fall into three buckets: dataset scale (the in-CI fixture is small, so production behavior may differ), hardware (CPU-only results may not match GPU rank order), and baseline coverage (we compared against the most directly comparable methods, not against every method in the literature).

A second class of limitation is methodological. Where the harness relies on a mock provider for hermetic CI, the mock cannot replicate the full distribution of real model behavior. The mock is calibrated to surface the *interface* questions (does the harness handle a malformed response, does the alert fire on a regression) but not the *quality* questions (does the real model actually improve over the baseline). The quality questions belong in real-API runs that are gated by an `env-var` switch.

A third class of limitation is scope. The harness deliberately ignores adjacent concerns (training, large-scale serving, multi-modal inputs); those belong in dedicated sibling projects in the same

portfolio. Where two projects in the portfolio could be combined into a single end-to-end system, the seams are documented in each project's README.

Finally, the harness assumes a competent operator. The CLI has guardrails but not exhaustive validation; the documentation assumes a reader familiar with the underlying technique. Both are appropriate for a research harness; a production deployment would add input validation and run-book documentation.

11 10. Future Work

The follow-up list is intentionally short and specific. Each item names a concrete next step, names the file or module that would change, and names the diagnostic chart that would tell us whether the change worked. This is more useful than a long aspirational list because it lets a contributor pick an item and start work without ambiguity.

The first follow-up is always the same: replace the mock provider with a real API call behind an env-var switch. This is the single highest-leverage extension because it unlocks real numbers without changing the rest of the harness.

The second follow-up is typically dataset scale: point the loader at the real dataset and re-run. This is documented in the README's Real . . . data section.

Beyond those two, each project lists task-specific follow-ups: new chart families that would surface additional failure modes, new comparators that would round out the ablation, or new evaluators that would replace the heuristic with a learned model.

- External queue with deadline-based eviction instead of admission-time rejection.
- Speculative-decoding overlay (each active session can run multiple draft tokens per step).
- Multi-LoRA scheduling overlay (admission must consider adapter cache hit rate).
- GPU calibration constants so absolute numbers approximate a specific deployment.
- Real-trace replay loader for production workload validation.

12 11. References

1. Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., Chun, B.-G. (2022). *Orca: A Distributed Serving System for Transformer-Based Generative Models*.
2. Kwon, W., Li, Z., Zhuang, S., et al. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention (vLLM)*.
3. Agrawal, A., Panwar, A., Mohan, J., Kwatra, N., Tumanov, A., Ramjee, R. (2023). *SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills*.
4. Lazowska, E. D. et al. (1984). *Quantitative System Performance*.

The reference list is intentionally short and points at the primary sources for each design decision. Secondary citations are in source-code docstrings where they belong; the report's reference list is

for the canonical papers a reader should consult to understand the technique.

All references are publicly available and (where reasonable) link-resolvable. Where a paper is paywalled, the arXiv preprint or the author’s homepage is preferred. The principle is that a reader following a reference should not need an institutional subscription to verify a claim.

13 Appendix A. Reproducibility Checklist

- ☒ MIT-licensed code.
- ☒ Workload seed-deterministic.
- ☒ Each scheduling strategy unit-tested.
- ☒ CI runs the full bench at smoke scale on every push.

14 Appendix B. Glossary

- **Continuous batching.** Per-step admission/retirement (Orca, vLLM).
- **Chunked prefill.** Interleave prefill chunks with decode (Sarathi).
- **KV budget.** Memory cap on the KV cache.
- **Admission control.** Reject (or queue) requests that exceed budget.